

CV-Tools — Référence des filtres

Chaque filtre, paramètre et mise en garde en un seul document

Référence des filtres

Chaque filtre est une simple fonction qui prend une image en premier argument et renvoie une nouvelle image. Les noms de registre (colonne name) sont ceux qui apparaissent dans les préférences JSON et les rapports.

Nom dans le registre	Fonction	Module	Sprint
clahe	apply_clahe	src.filters.clahe	1
contrast_brightness	adjust_contrast_brightness	src.filters.contrast_brightness	
auto_contrast	auto_contrast	src.filters.contrast_brightness	
levels	adjust_levels	src.filters.levels	1
auto_levels	auto_levels	src.filters.levels	1
histeq	histogram_equalization	src.filters.histogram_equalization	
roi_crop	roi_crop	src.filters.roi	1
roi_draw	roi_draw	src.filters.roi	1
crop	crop	src.filters.crop_resize	1
resize	resize	src.filters.crop_resize	1
rotate	rotate	src.filters.crop_resize	1
flip	flip	src.filters.crop_resize	1
sharpen	unsharp_mask	src.filters.sharpen	2
sharpen_laplacian	laplacian_sharpen	src.filters.sharpen	2
gaussian_blur	gaussian_blur	src.filters.smoothing	2
median_filter	median_filter	src.filters.smoothing	2
bilateral_filter	bilateral_filter	src.filters.smoothing	2
canny	canny_edges	src.filters.edge_detection	
auto_canny	auto_canny	src.filters.edge_detection	
sobel	sobel_edges	src.filters.edge_detection	
laplacian	laplacian_edges	src.filters.edge_detection	

<code>ela</code>	<code>error_level_analysis</code>	<code>src.filters.ela</code>	3
<code>fft_spectrum</code>	<code>fft_magnitude_spectrum</code>	<code>src.filters.fft_analysis3</code>	
<code>fft_filter</code>	<code>fft_filter</code>	<code>src.filters.fft_analysis3</code>	
<code>remove_periodic</code>	<code>remove_periodic_noise</code>	<code>src.filters.fft_analysis3</code>	
<code>noise_map</code>	<code>noise_map</code>	<code>src.filters.noise_analysis3</code>	
<code>clone_detect</code>	<code>highlight_clones</code>	<code>src.filters.clone_detection</code>	
<code>deblur_motion</code>	<code>deblur_motion</code>	<code>src.filters.motion_deblur3</code>	
<code>deblur_defocus</code>	<code>deblur_defocus</code>	<code>src.filters.motion_deblur3</code>	
<code>curves / s_curve</code>	<code>apply_curve / s_curve</code>	<code>src.filters.curves</code>	—
<code>white_balance / white_balance_patch / temperature</code>	<code>auto_white_balance / white_balance_from_patch / adjust_temperature</code>	<code>src.filters.white_balance</code>	—
<code>saturation / vibrance / desaturate / selective_saturation</code>	voir le module	<code>src.filters.saturation</code>	—
<code>color_balance / cmyk / channel_mixer</code>	voir le module	<code>src.filters.color_balance</code>	—
<code>invert / invert_channel / invert_luminance / solarize</code>	voir le module	<code>src.filters.invert</code>	—
<code>nl_means / nl_means_auto</code>	<code>nl_means_denoise</code>	<code>src.filters.nl_means_denoise</code>	
<code>upscale</code>	<code>upscale</code>	<code>src.filters.super_resolution</code>	
<code>local_contrast / detail_enhance / multiscale_detail / texture_boost</code>	voir le module	<code>src.filters.detail_enhancement</code>	
<code>perspective / auto_perspective</code>	<code>correct_perspective</code>	<code>src.filters.perspective_correction</code>	
<code>barrel / fisheye</code>	<code>correct_barrel_distortion / correct_fisheye</code>	<code>src.filters.fisheye_correction</code>	

<code>pixel_aspect / fit_aspect</code>	<code>correct_pixel_aspect / fit_to_aspect</code>	<code>src.filters.aspect_ratio—</code>
<code>undistort</code>	<code>undistort_with_file</code>	<code>src.filters.undistort</code> —
<code>blocking_map / deblock</code>	voir le module	<code>src.filters.compression_analysis</code>
<code>stain</code>	<code>extract_stain</code>	<code>src.filters.color_deconvolution</code>
<code>component / bit_plane</code>	<code>extract_component / extract_bit_plane</code>	<code>src.filters.component_separation</code>
<code>redact</code>	<code>redact_region</code>	<code>src.filters.redaction</code> —

Sprint 1 — Ajuster & Corriger

CLAHE — `clahe`

Amélioration adaptative du contraste appliquée par tuiles, avec une limitation du contraste pour éviter d'amplifier le bruit. Le filtre de référence pour les images de vidéosurveillance sombres ou à faible contraste.

Paramètre	Type	Défaut	Remarques
<code>clip_limit</code>	float	2.0	Plus élevé = plus de contraste local, plus de bruit
<code>tile_grid_size</code>	int ou (lignes, colonnes)	8	8 signifie des tuiles de 8×8
<code>color_mode</code>	str	'lab'	lab, hsv, yuv, channelwise, luminance

lab, hsv et yuv égalisent un seul canal de luminance et laissent la chrominance inchangée, la couleur reste donc stable. channelwise égalise R, G et B indépendamment et **modifie** la couleur — à n'utiliser que si c'est l'effet recherché.

CLI: `--clahe clip=3.0 tile=8x8 mode=lab`

`apply_clahe_grid(image, clip_limits, tile_grid_sizes)` génère une grille annotée de combinaisons de paramètres pour choisir des valeurs rapidement.

Contraste & Luminosité — `contrast_brightness`

$\text{sortie} = (\text{entrée} - 128) * \text{contraste} + 128 + \text{luminosité}$, puis gamma.

Paramètre	Type	Défaut	Remarques
<code>brightness</code>	float	0.0	Décalage, de -255 à 255
<code>contrast</code>	float	1.0	1.0 = inchangé
<code>gamma</code>	float	1.0	<1 assombrit les tons moyens, >1 les éclaircit
<code>channel</code>	str ou None	None	r, g, b pour cibler un seul canal

CLI: `--brightness 30, --contrast 1.5, --gamma 0.8` (chacun est une étape distincte de la chaîne)

Contraste automatique — `auto_contrast`

Étire l'histogramme de luminance sur toute la plage.

Paramètre	Type	Défaut	Remarques
<code>cutoff</code>	float	0.0	Pourcentage des pixels les plus sombres/clairs à ignorer (0–50)

CLI : `--auto-contrast` ou `--auto-contrast 2`

Niveaux — `levels`

Fait correspondre la plage d'entrée [`black_point`, `white_point`] à la plage de sortie [`output_black`, `output_white`] avec une courbe gamma sur les tons moyens.

Paramètre	Type	Défaut
<code>black_point</code>	float	0
<code>gamma</code>	float	1.0
<code>white_point</code>	float	255
<code>output_black</code>	float	0
<code>output_white</code>	float	255
<code>channel</code>	str ou None	None

Lève une `ValueError` si `black_point >= white_point`.

CLI : `--levels 20,1.0,220` (noir, gamma, blanc)

Niveaux automatiques — `auto_levels`

Paramètre	Type	Défaut	Remarques
<code>per_channel</code>	bool	False	True étire R/G/B indépendamment et peut altérer la couleur

CLI : `--auto-levels`

Égalisation d'histogramme — `histeq`

Égalisation globale — aplatit tout l'histogramme d'un coup. Plus forte et plus brutale que CLAHE ; a tendance à amplifier le bruit dans les zones plates.

Paramètre	Type	Défaut	Remarques
color_mode	str	'lab'	lab, hsv, yuv, channelwise, grayscale
mask	ndarray ou None	None	Restreint la région utilisée pour construire l'histogramme

CLI : `--histeq mode=lab`

Recadrage ROI — roi_crop

Recadre sur une région, **limitée** aux bords de l'image — une région surdimensionnée se réduit silencieusement.

Paramètre	Type
x, y, width, height	int

CLI : `--roi 100,100,300,200`

Dessin ROI — roi_draw

Dessine un rectangle pour marquer une région sans altérer les pixels en dessous (sauf si `filled`).

Paramètre	Type	Défaut
x, y, width, height	int	—
color	(r, g, b)	(255, 0, 0)
thickness	int	2
label	str ou None	None
filled	bool	False
alpha	float	0.3

CLI : `--draw-roi 265,295,120,46`

Recadrage — crop

Même géométrie que `roi_crop`, mais **lève** une `ValueError` lorsque la région tombe entièrement hors de l'image, au lieu de renvoyer un résultat tronqué. À utiliser quand un recadrage hors limites doit être une erreur.

CLI : `--crop 100,100,300,200`

Redimensionnement — `resize`

Paramètre	Type	Remarques
<code>width</code>	int ou None	Seul, la hauteur suit le ratio d'aspect
<code>height</code>	int ou None	Seule, la largeur suit le ratio d'aspect
<code>scale</code>	float ou None	Utilisé en l'absence de width/height
<code>interpolation</code>	str	<code>auto</code> , <code>nearest</code> , <code>bilinear</code> , <code>bicubic</code> , <code>lanczos</code> , <code>area</code>

`auto` choisit `INTER_AREA` pour la réduction et `INTER_LANCZOS4` pour l'agrandissement. Pour un travail forensique, préférer `nearest` quand il faut inspecter les pixels sans rééchantillonnage.

CLI : `--resize 800x600`, `--resize 800x`, `--resize x600`, `--resize 50%`, `--resize 0.5`, combiné avec `--interpolation lanczos`.

Rotation — `rotate`

Agrandit le canevas pour qu'aucun contenu ne soit tronqué.

Paramètre	Type	Défaut
<code>angle</code>	float	— (degrés, sens antihoraire)
<code>center</code>	(x, y) ou None	<code>None</code> = centre de l'image
<code>scale</code>	float	<code>1.0</code>
<code>border_mode</code>	str	<code>'constant'</code> (<code>replicate</code> , <code>reflect</code> , <code>wrap</code>)
<code>border_value</code>	(r, g, b)	<code>(0, 0, 0)</code>

CLI : `--rotate 90`

Retournement — `flip`

Paramètre	Type	Valeurs
<code>direction</code>	str	<code>horizontal</code> , <code>vertical</code> , <code>both</code>

CLI : `--flip horizontal`

Sprint 2 — Améliorer

Masque flou (Unsharp Mask) — `sharpen`

Soustrait une copie floutée pour isoler le détail, puis le rajoute pondéré par `amount`.

Paramètre	Type	Défaut	Remarques
<code>amount</code>	float	1.0	0 = aucun changement ; au-delà de 2, l'effet paraît généralement artificiel
<code>radius</code>	float	1.0	Sigma du flou. Petit = détail fin, grand = contraste local.
<code>threshold</code>	int	0	Contraste local minimal (0–255) avant qu'un pixel ne soit accentué

`threshold` est le contrôle du bruit : l'augmenter laisse tranquilles les zones lisses — là où vit le bruit — tout en accentuant les véritables contours. Accentuer **après** le débruitage, jamais avant.

CLI : `--sharpen amount=1.5 radius=1.0 threshold=4`

`sharpen_grid(image, amounts, radii)` génère une grille annotée de paramètres, comme `apply_clahe_grid`.

Accentuation laplacienne — `sharpen_laplacian`

`sortie = entrée - force × laplacien(entrée)`. Plus dur et plus sensible au bruit qu'un masque flou, mais ne nécessite pas de choisir un rayon.

Paramètre	Type	Défaut
<code>strength</code>	float	1.0
<code>kernel_size</code>	int	3 (impair, 1–31)

CLI : `--sharpen-laplacian strength=1.0 kernel=3`

Flou gaussien — `gaussian_blur`

Paramètre	Type	Défaut	Remarques
<code>radius</code>	float	2.0	Sigma gaussien en pixels

<code>kernel_size</code>	int	0	0 dérive le noyau à partir du rayon — c'est normalement ce qu'il faut
--------------------------	-----	---	---

Lissage généraliste. Floute les contours en même temps que le bruit ; préférer le bilatéral quand les contours importent.

CLI : `--gaussian 1.5` (`--gaussian` seul utilise 2.0)

Filtre médian — `median_filter`

Paramètre	Type	Défaut	Remarques
<code>kernel_size</code>	int	3	Impair, 3 ou plus

Le remède standard contre le bruit impulsionnel (poivre et sel). La sortie ne contient jamais que des valeurs déjà présentes à proximité, donc les plateaux et les contours survivent intacts là où un flou les aurait étalés.

CLI : `--median 3` (`--median` seul utilise 3)

Filtre bilatéral — `bilateral_filter`

Paramètre	Type	Défaut	Remarques
<code>diameter</code>	int	9	Diamètre du voisinage ; 0 le dérive de <code>sigma_space</code>
<code>sigma_color</code>	float	75.0	Tolérance à la différence de couleur
<code>sigma_space</code>	float	75.0	Étendue spatiale

Pondère les voisins à la fois par la distance et par la similarité de couleur, ce qui permet de lisser le bruit du capteur à l'intérieur des régions sans déborder sur les frontières. Le plus lent des trois filtres de lissage.

CLI : `--bilateral d=9 color=75 space=75`

Sprint 2 — Analyser

Les quatre détecteurs de contours renvoient tous une carte **mono-canal** en uint8, donc ils convertissent une image couleur en niveaux de gris en cours de chaîne.

Canny — `canny`

Paramètre	Type	Défaut	Remarques
<code>low_threshold</code>	float	100	Les contours faibles ne survivent que s'ils sont connectés à un contour fort
<code>high_threshold</code>	float	200	Un ratio bas:haut de 1:2 ou 1:3 est le point de départ habituel
<code>aperture_size</code>	int	3	3, 5, ou 7
<code>l2_gradient</code>	bool	False	Magnitude L2 exacte plutôt que l'approximation L1, moins coûteuse
<code>blur_sigma</code>	float	0.0	Flou gaussien préalable — nécessaire sur des images bruitées

La sortie est binaire (0 ou 255).

CLI : `--canny 50,150`, avec `--blur-first 1.5` pour régler le flou préalable.

Auto Canny — `auto_canny`

Dérive les deux seuils à partir de l'intensité médiane de l'image, ce qui convient à des lots d'images dont l'exposition varie. Revient à 50/150 lorsque la médiane fait coïncider les deux seuils (une image quasi noire ou quasi blanche).

Paramètre	Type	Défaut	Remarques
<code>sigma</code>	float	0.33	Écart autour de la médiane, 0–1. Plus grand conserve plus de contours.
<code>blur_sigma</code>	float	0.0	Flou gaussien préalable

CLI : `--auto-canny` ou `--auto-canny 0.4`

Sobel — `sobel`

Paramètre	Type	Défaut	Remarques
<code>dx</code>	int	1	Ordre de la dérivée horizontale (0 ou 1)
<code>dy</code>	int	1	Ordre de la dérivée verticale (0 ou 1)
<code>kernel_size</code>	int	3	Impair, 1–7
<code>normalize</code>	bool	True	Étire le résultat pour remplir 0–255

Avec `dx` et `dy` réglés tous les deux, on obtient la magnitude du gradient ; avec un seul, on obtient cette dérivée directionnelle unique. Valeurs continues, contrairement à la sortie binaire de Canny.

CLI : `--sobel dx=1 dy=1 kernel=3`

Laplacien — `laplacian`

Paramètre	Type	Défaut
<code>kernel_size</code>	int	3 (impair, 1–31)
<code>normalize</code>	bool	True
<code>blur_sigma</code>	float	0.0

Répond au changement d'intensité dans toutes les directions à la fois — et au bruit tout aussi volontiers, donc un léger `blur_sigma` vaut généralement le coup.

CLI : `--laplacian kernel=3 blur=1.0`

Histogramme (pas une étape de chaîne)

`histogram.py` analyse plutôt qu'il ne transforme, il est donc exposé sur la CLI comme des options de sortie plutôt que comme des filtres du registre.

Fonction	Renvoie
<code>compute_histogram(image, bins=256, normalize=False)</code>	Dictionnaire nom de canal → comptes par bin
<code>histogram_stats(image)</code>	Moyenne, médiane, écart-type, min, max, p1, p99, % d'écrtage par canal

<code>dynamic_range_used(image)</code>	Fraction de 0–255 couverte entre p1 et p99
<code>render_histogram(image, ...)</code>	Image du graphique en RVB

Les pourcentages d'écrêtage sont la partie forensiquement importante : les pixels bloqués à 0 ou 255 ont perdu leurs valeurs d'origine, et aucune amélioration ne les récupère. Un faible `dynamic_range_used` signifie que les niveaux ou le CLAHE ont encore de la marge.

`render_histogram` accepte `width`, `height`, `bins`, `log_scale` (révèle les queues éparses qu'un tracé linéaire aplatit à néant), `show_grid`, `background`, et `channels` pour restreindre les courbes tracées.

CLI : `--histogram chart.png`, `--histogram-log`, `--hist-stats`

`edge_density(edges, threshold=0)` donne la fraction de pixels porteurs d'un contour. Elle compare la netteté entre images d'une même scène, mais ce n'est pas une mesure générale de netteté — flouter une image dont la seule caractéristique est un contour marqué étale ce contour sur plus de pixels et augmente la densité.

Sprint 3 — Forensique

À lire avant d'utiliser l'un de ces filtres. Ils repèrent des éléments *qui méritent d'être examinés*. Aucun d'eux n'établit qu'une image a été manipulée, et chacun a un mode d'échec qui produit une preuve d'apparence convaincante pour une conclusion fausse. Les mises en garde ci-dessous font partie intégrante de l'outil, ce ne sont pas de simples avertissements autour de lui.

Analyse du niveau d'erreur (ELA) — `ela`

Recompresse l'image en JPEG et amplifie la différence. Une région dont l'historique de compression diffère de son entourage peut présenter un niveau d'erreur différent.

Paramètre	Type	Défaut	Remarques
<code>quality</code>	int	90	Qualité de recompression, 1–100. Viser une valeur proche de l'originale.
<code>scale</code>	float	0	Multiplicateur de luminosité ; 0 met à l'échelle automatiquement pour que le pic d'erreur atteigne 255
<code>grayscale</code>	bool	False	Regroupe l'erreur par canal en un seul canal

Limites. Significatif uniquement sur un original JPEG — un réenregistrement en PNG, ou un second enregistrement JPEG complet de l'image, efface totalement le signal. Les zones claires suivent la densité de contours et de texture autant que l'historique d'édition, donc les régions chargées paraissent toujours « chaudes ». Une carte propre ne signifie pas que l'image est authentique.

CLI : `--ela quality=90 gray=true, --ela-stats [QUALITY]`

`ela_stats(image, quality, block_size)` renvoie l'erreur moyenne/maximale, une grille de moyennes par bloc, et le bloc le plus chaud avec un score z indiquant à quel point il se distingue du reste. `recompress(image, quality)` expose seul l'aller-retour JPEG.

Spectre de magnitude FFT — `fft_spectrum`

Paramètre	Type	Défaut	Remarques
<code>log_scale</code>	bool	True	Sans cela, le terme DC écrase tout et le tracé n'est qu'un point
<code>normalize</code>	bool	True	Étire pour remplir 0–255

Le DC est au centre. Les structures périodiques — entrelacement, trames de similitude, bandes de scanner — apparaissent sous forme de points lumineux discrets éloignés de ce centre.

CLI : `--fft log=true`

Filtre fréquentiel — `fft_filter`

Paramètre	Type	Défaut	Remarques
<code>filter_type</code>	str	'lowpass'	lowpass, highpass, bandpass
<code>cutoff</code>	float	30.0	Rayon en pixels depuis le DC
<code>cutoff_high</code>	float	0.0	Rayon supérieur, bandpass uniquement
<code>soft</code>	bool	True	Masque à bord gaussien. Un bord dur provoque un effet de sonnerie qui ressemble à un vrai contenu d'image.

Renvoie une sortie mono-canal.

CLI : `--fft-filter type=highpass cutoff=20`

Suppression de bruit périodique — `remove_periodic`

Trouve les pics spectraux isolés et les élimine par filtre coupe-bande, supprimant un motif répétitif avec bien moins de dégâts qu'un flou. Le pic miroir est également éliminé, car le spectre d'une image réelle est symétrique par rapport au DC.

Paramètre	Type	Défaut	Remarques
<code>peaks</code>	list ou None	None	DéTECTÉS automatiquement si omis
<code>notch_radius</code>	float	4.0	Rayon du coupe-bande gaussien sur chaque pic
<code>min_radius</code>	float	10.0	Ignore ce rayon autour du DC — les basses fréquences sont l'image elle-même
<code>threshold</code>	float	4.0	Écarts-types au-dessus du fond local pour qu'un pic compte

Un résidu du motif survit généralement sur les bords de l'image : la FFT traite l'image comme si elle pavait le plan, et la discontinuité aux bords disperse de l'énergie dans tout le spectre.

CLI : `--remove-periodic notch=4`

`detect_periodic_peaks(image, ...)` renvoie les pics seuls, pour les inspecter avant l'élimination.

Carte de bruit — `noise_map`

Paramètre	Type	Défaut	Remarques
<code>block_size</code>	int	32	Plus petit localise mieux mais estime chaque bloc moins fiablement
<code>normalize</code>	bool	True	Étire pour remplir 0–255
<code>upscale</code>	bool	True	Redimensionne la grille de blocs aux dimensions de l'image d'entrée

Plus clair signifie plus bruité. Le bruit du capteur devrait être assez uniforme sur une image non retouchée, donc une région qui diffère nettement provient d'ailleurs — un capteur différent, un redimensionnement différent, ou un débruitage appliqué à cette seule région. Une forte texture augmente aussi la mesure.

CLI : `--noise-map 32, --noise-stats`

`estimate_noise(image)` renvoie le sigma global via la méthode d'Immerkaer ; `estimate_snr` donne un résultat en dB (en utilisant l'écart-type de l'image elle-même comme signal, donc une image plate obtient un score bas même si elle est propre) ; `noise_report` ajoute des statistiques par bloc et un ratio d'`uniformity`.

Détection de clonage — `clone_detect`

Trouve les régions dupliquées d'ailleurs dans la même image. Les blocs sont décrits par leurs coefficients DCT de basse fréquence, triés de sorte que les blocs quasi identiques deviennent voisins, et un décalage partagé par de nombreuses paires est signalé comme une région dupliquée.

Paramètre	Type	Défaut	Remarques
<code>step</code>	int	1	Seuls les décalages multiples de cette valeur sont détectables
<code>block_size</code>	int	16	Côté de chaque bloc comparé
<code>coefficients</code>	int	4	Taille du carré supérieur gauche de coefficients DCT conservé comme descripteur

<code>quantization</code>	float	<code>4.0</code>	Arrondi du descripteur ; plus grand tolère plus de compression, mais produit plus de fausses correspondances
<code>min_distance</code>	float	<code>0</code>	Séparation minimale pour qu'une paire compte ; 0 utilise 2x <code>block_size</code>
<code>min_matches</code>	int	<code>8</code>	Nombre de paires requis avant qu'un décalage soit signalé
<code>min_variance</code>	float	<code>12.0</code>	Les blocs sans relief en dessous de ce seuil sont ignorés
<code>max_blocks</code>	int	<code>300000</code>	Garde-fou mémoire ; lève une erreur plutôt que d'allouer des gigaoctets

La contrainte `step` est celle qui piège le plus souvent. Les blocs sont échantillonnés sur une grille de ce pas, donc une région déplacée de 190 pixels est invisible pour un pas de 8 — la copie est échantillonnée à une phase différente de l'original et les descripteurs ne correspondent pas. La valeur par défaut de 1 est exhaustive. L'augmenter donne un passage de dépistage rapide, pas une recherche complète.

Sur une grande image, recadrer sur une région avec `--roi` plutôt que d'augmenter `step`.

Limites. Une répétition authentique — mur de briques, fenêtres, carrelage, texte — est de la duplication, et sera signalée. Cet outil localise la duplication, pas l'intention.

CLI : `--clone-detect block=16 step=1 matches=8 variance=12, --clone-stats`

`detect_copy_move` renvoie le dictionnaire de résultat complet (masque, vecteurs de décalage, comptes de blocs) ; `draw_clone_regions` le superpose en teinte sur l'image.

Défloutage de Wiener — `deblur_motion`, `deblur_defocus`

Inverse un flou connu. L'inversion naïve divise par la réponse fréquentielle du flou, qui est proche de zéro à certaines fréquences, ce qui amplifie le bruit à ces fréquences sans limite. Le filtre de Wiener tempère cela : $F = G \cdot \text{conj}(H) / (|H|^2 + K)$, où `K` est `noise_power`.

Paramètre	Type	Défaut	Remarques
<code>length</code>	float	<code>15.0</code>	Étendue du mouvement en pixels (<code>deblur_motion</code>)
<code>angle</code>	float	<code>0.0</code>	Direction du mouvement en degrés, 0 = horizontal (<code>deblur_motion</code>)

<code>radius</code>	float	5.0	Rayon du cercle de défocalisation (<code>deblur_defocus</code>)
<code>noise_power</code>	float	0.01	Augmenter sur des images bruitées pour échanger de la netteté contre de la stabilité

L'entrée est complétée en miroir (reflect-padding) avant la transformée puis recadrée après, ce qui écarte du résultat le repliement (wraparound) de la FFT — où le bord gauche est convolué avec le bord droit.

Limites. Il faut fournir la PSF correcte ; une longueur ou un angle devinés produisent un détail d'apparence confiante qui n'a jamais été enregistré. La déconvolution suppose aussi un flou uniforme sur toute l'image, donc une scène où un seul objet a bougé nécessite d'isoler d'abord cet objet avec `--roi`.

CLI : `--deblur length=15 angle=30 noise=0.01, --deblur-defocus radius=5 noise=0.01`

`motion_blur_psf` et `defocus_psf` construisent les PSF ; `apply_psf` en applique une vers l'avant, utile pour prévisualiser ce que signifie une PSF. `wiener_deconvolution` accepte une PSF arbitraire.

Comme la véritable PSF ne peut être lue de manière fiable sur une image, `deblur_sweep(image, lengths, angles)` génère une grille annotée sur l'espace des paramètres pour juger à l'œil, et `focus_score(image)` classe les résultats selon la variance du laplacien.

Sprint 3 — Multi-images (pas des étapes de chaîne)

`frame_averaging.py` consomme une *séquence* d'images et produit une seule image, il s'exécute donc avant la chaîne de filtres plutôt qu'à l'intérieur. Sur la CLI, c'est `--frames N`, avec `--frame` pour sélectionner l'index de départ et `--frame-step` pour le pas.

Fonction	Méthode CLI	Ce qu'elle fait
<code>average_frames(frames, weights=None)</code>	<code>mean</code>	Supprime le bruit aléatoire ; le bruit diminue avec la racine carrée du nombre d'images
<code>median_frames(frames)</code>	<code>median</code>	Supprime tout ce qui est présent dans moins de la moitié des images, reconstruisant l'arrière-plan
<code>integrate_frames(frames, gain, auto_scale)</code>	<code>integrate</code>	Accumule la lumière d'images très sombres sans amplifier le bruit comme le ferait un gain
<code>sharpest_frames(frames, count)</code>	<code>sharpest</code>	Classe les images par netteté ; la CLI moyenne la meilleure moitié

Toutes supposent que les images sont **alignées**. Des images filmées à la main ou avec une caméra PTZ nécessitent une stabilisation préalable — une caméra en mouvement transforme le moyennage en flou.

`frame_difference(a, b, amplify)` donne la différence absolue entre deux images, pour isoler ce qui a bougé.

CLI : `--frames 24 --frame-method median --frame-step 5`

Catalogue restant

Le docstring de chaque module porte le raisonnement complet ; voici le résumé et les mises en garde les plus importantes.

Ajuster

curves — courbe tonale à points de contrôle, interpolée avec une spline monotone (PCHIP) de sorte que la correspondance ne puisse jamais rebrousser chemin et inverser l'ordre tonal comme peut le faire une cubique ordinaire. Préréglages : `linear`, `brighten`, `darken`, `contrast`, `reduce_contrast`, `lift_shadows`, `film`. CLI : `--curves preset=lift_shadows` ou `--curves points=0:0,128:170,255:255`.

white_balance — chaque méthode automatique suppose quelque chose sur la scène, et échoue quand ce n'est pas vérifié : `gray_world` (la moyenne est neutre — échoue quand une couleur domine), `white_patch` (les pixels les plus clairs sont blancs — échoue sur une zone surexposée), `shades_of_gray` (un compromis, et l'option par défaut). Quand la scène contient un élément connu pour être neutre, `--wb-patch X,Y,W,H` le mesure au lieu de deviner.

saturation / vibrance — la vibrance pondère l'augmentation vers les couleurs peu saturées afin que les couleurs vives ne se figent pas en aplats. À noter que l'atténuation est *proportionnelle* : une couleur à saturation moyenne peut encore gagner plus de saturation brute qu'une couleur presque neutre.

color_balance — décale les ombres, les tons moyens et les hautes lumières indépendamment, avec des pondérations gaussiennes qui se chevauchent pour que les plages se fondent plutôt que de créer des bandes. Utile quand deux sources lumineuses colorent différemment selon la luminosité. `preserve_luminosity` maintient la luminosité globale fixe pour que seule la couleur change.

invert — `invert_luminance` inverse la luminosité tout en conservant la teinte, ce qui rend parfois lisible un détail sombre sur fond sombre là où augmenter la luminosité ne fait que le délayer.

Améliorer

nl_means — moyenne les patchs qui se *ressemblent* où qu'ils se trouvent, de sorte qu'une texture répétitive se renforce plutôt qu'elle ne se lisse. Le débruiteur le plus lent de la boîte à outils ; le coût croît avec le carré de `search_window`. Régler `h` à partir du bruit mesuré via `estimate_h`, ou utiliser `--nl-means-auto`.

`nl_means_denoise_frames` utilise les images voisines comme preuve supplémentaire, sans le flou de mouvement que provoque le simple moyennage d'images sur les objets en mouvement.

super_resolution — la distinction ici compte plus que partout ailleurs dans la boîte à outils. `upscale` **interpole et n'ajoute aucune information** ; une plaque illisible à la résolution native reste illisible agrandie. `super_resolve` récupère véritablement du détail, car le mouvement infra-pixellaire entre les images échantillonne la scène sur des grilles différentes. Cela nécessite un vrai mouvement infra-pixellaire — `super_resolve_report` indique si une séquence en contient.

La corrélation de phase pilote l'alignement, et elle nécessite du détail à large bande. Une scène fortement périodique (carrelage, briques, une clôture) produit plusieurs pics de corrélation de hauteur similaire, et le décalage mesuré peut être dénué de sens plutôt que simplement imprécis.

detail_enhancement — `local_contrast` est un masque flou à grand rayon, ce qu'est la plupart des curseurs « clarté ». `enhance_detail` préserve les contours, il fait donc ressortir la texture sans halos. `multiscale_detail` accentue les bandes de fréquence indépendamment.

Corriger

perspective — rectification à quatre points. Fournir le ratio réel connu de la surface (`KNOWN_RATIOS` : `a4_portrait`, `a4_landscape`, `us_letter`, `credit_card`, `plate_eu`, `plate_us`, `square`), car l'estimer à partir d'une vue en perspective n'est pas fiable. `find_document_corners` détecte automatiquement une surface rectangulaire ; elle renvoie `None` sur une scène encombrée, ce qui est le résultat attendu et non une erreur.

fisheye_correction — `barrel` utilise le modèle radial polynomial (réglé à la main, convient pour un grand angle modéré) ; `fisheye` utilise le modèle équidistant pour les véritables caméras dôme. Les deux *estiment* la distorsion. Quand la caméra est disponible, calibrer plutôt.

aspect_ratio — la vidéo SD n'utilise pas de pixels carrés ; affichée sans correction, tout est étiré et toute mesure est faussée dans un axe. `PIXEL_ASPECT_RATIOS` couvre le PAL, le NTSC, le HDV et l'anamorphique. Le mode `pad` de `fit_to_aspect` est le seul qui ne modifie ni la géométrie ni le contenu.

undistort — la voie la plus défendable : dériver les intrinsèques réelles de la caméra à partir de photographies d'un échiquier, puis inverser exactement cela. `CameraCalibration.is_reliable` vérifie que l'erreur de reprojection est inférieure à un pixel. Une calibration est spécifique à une caméra, un zoom et une mise au point donnés ; appliquer la calibration d'une autre caméra est pire que n'en appliquer aucune, car le résultat paraît plausible tout en étant géométriquement faux.

Analyser

compression_analysis — `blockiness_score` compare les sauts d'intensité sur la grille JPEG de 8 pixels à ceux observés ailleurs. `estimate_jpeg_quality` lit directement les tables de quantification d'un JPEG, ce qui est exact plutôt que déduit — mais seulement tant que le fichier reste un JPEG.

La mesure suppose un contenu photographique. Une image dominée par des contours synthétiques durs qui ne tombent pas sur la grille de blocs gonfle le terme intérieur et peut ne montrer aucun blocage quel que soit son historique. Un fort blocage signifie une compression importante, rien de plus ; un réenregistrement uniformise la grille et efface toute différence locale.

Spécial

color_deconvolution — sépare des colorants qui se superposent (encre sur imprimé, tampon sur signature) en résolvant dans le domaine de la densité optique, où l'absorption est additive. Au plus trois colorants, puisque trois canaux donnent trois équations, et les colorants aux vecteurs de couleur quasi parallèles se séparent mal. `estimate_stain_vector` mesure un vecteur à partir d'un échantillon d'un seul colorant, ce qui est la façon fiable de construire une séparation pour une encre inconnue.

component_separation — un détail invisible dans un composite couleur est souvent évident dans une seule composante. Espaces colorimétriques (`rgb`, `hsv`, `hls`, `lab`, `luv`, `ycrcb`, `yuv`, `xyz`), séparation fréquentielle (base contre détail), et plans de bits. La structure dans les plans de bits de poids faible est notable : le bruit naturel du capteur n'en a aucune, donc un motif à cet endroit suggère des données cachées ou une région collée.

redaction — la seule opération qui ne doit pas être réversible, et les méthodes évidentes échouent. **Le flou est réversible** — c'est une convolution connue, et la déconvolution de Wiener de cette même boîte à outils peut l'annuler. **La pixellisation est réversible pour un texte court à alphabet connu** — rendre chaque plaque candidate et faire correspondre les moyennes de blocs est une attaque documentée et peu coûteuse. Seuls `fill` et `noise` écartent réellement les pixels d'origine ; `fill` est la méthode par défaut et la seule à utiliser pour un document destiné à être publié. `verify_redaction` corréle chaque région avec l'original et indique si le contenu a réellement disparu.

`annotate` — flèches, formes, texte, et mesure calibrée. `scale` convertit des pixels en unités ; `measure_distance` (1D), `measure_area` (2D, formule du lacet), `draw_measurement` et `draw_scale_bar` les présentent.

Une échelle n'est valide que pour le plan dans lequel elle a été mesurée. Une règle posée au sol calibre les distances au sol et ne dit rien d'un panneau trois mètres plus loin, qui est plus éloigné de la caméra et donc plus petit par pixel. Corriger d'abord la perspective. Les annotations sont dessinées sur une copie — conserver l'original non annoté, car une image annotée est une figure, pas une preuve.

`measure_3d` — la hauteur hors du plan du sol, que l'échelle ci-dessus ne peut pas atteindre. Étant donné l'horizon du plan du sol, le point de fuite des verticales de la scène, et un objet de référence de hauteur connue posé sur ce sol, la hauteur de tout autre objet posé sur le même sol découle d'un rapport anharmonique (cross-ratio) — une quantité que la projection préserve. La méthode est celle de Criminisi, Reid et Zisserman, *Single View Metrology*, IJCV 40(2), 2000.

Paramètre	Défaut	Signification
<code>base_top</code>	requis	Point de contact au sol et point le plus haut de la cible
<code>reference_base</code> , <code>reference_top</code>	requis	Les deux mêmes points sur l'objet connu
<code>horizon</code>	requis	Une ligne <code>y</code> pour une caméra de niveau, <code>x1, y1, x2, y2</code> , ou <code>a, b, c</code>
<code>reference_height</code>	<code>1800.0</code>	Hauteur réelle de la référence, dans <code>unit_name</code>
<code>vertical_point</code>	<code>None</code>	Point de fuite vertical ; à omettre si les verticales sont parallèles
<code>unit_name</code>	<code>'mm'</code>	Libellé de l'unité
<code>show_horizon</code>	<code>True</code>	Dessine l'horizon sur lequel repose l'estimation

`measure_height` renvoie le nombre sans dessiner, ainsi que `uncertainty_per_pixel` — de combien la réponse varie pour une erreur d'un pixel sur les points cliqués. À lire avant de citer une hauteur ; c'est le plancher de l'erreur, pas l'erreur totale. `vanishing_point` résout un point de fuite à partir de deux lignes parallèles de la scène ou plus, par moindres carrés, et `horizon_from_vanishing_points` construit l'horizon à partir de deux d'entre eux.

L'estimation ne vaut que ce que valent ses hypothèses, et chacune échoue silencieusement :

- **Les deux objets doivent se tenir sur le même plan de sol.** Quelqu'un sur un trottoir, une marche ou une pente est mesuré par rapport à un plan sur lequel il ne se trouve pas.
- **Corriger d'abord la distorsion de l'objectif.** Des lignes droites courbées faussent la géométrie de fuite avant même le début du calcul.
- **La base est le point de contact au sol.** Un espace sous le talon, ou des pieds cachés derrière une voiture, biaise directement le résultat.
- **La précision s'effondre près de l'horizon**, où un seul pixel couvre une distance réelle qui croît rapidement — ce que rapporte justement `uncertainty_per_pixel`.
- **Omettre `vertical_point` suppose que la caméra n'a aucune inclinaison.** Sur une caméra synthétique à 2,5 m de hauteur avec une référence de 1,8 m, cette hypothèse a surestimé d'environ 16 mm pour 5°

d'inclinaison et de 33 mm pour 18°. La plupart des caméras de vidéosurveillance sont inclinées, donc fournir le point de fuite vertical.

Fonctions d'aide ROI (pas des étapes de chaîne)

`analyze_roi(image, roi)` renvoie la moyenne, l'écart-type, le min et le max par canal, plus le nombre de pixels. Exposé sur la CLI comme `--analyze-roi X,Y,W,H`, qui s'exécute sur l'image **traitée**.

`apply_to_roi(image, roi, filter_fn, **kwargs)` applique n'importe quel filtre à une région seulement, en laissant le reste de l'image inchangé.

`get_centered_roi(shape, w, h)` et `roi_from_ratio(shape, x, y, w, h)` construisent des régions sans coder en dur des coordonnées en pixels.